The full cell system (D43–D50)

Status: stages A–G landed as D44, D45, D46, D47, D48, D49, and D50. This note carries the design and what landed. A domain that does not use the cell system is unaffected.

The term and the surface

Use **class** for the cell-system concept. It is not the old `ClassTag` classification verdict: that is now `ValueTag`, and the word is `value-of`. The code type for a named value set remains `ValueSet`; a class's extension is a value set, and the class definition is its nominal cell metadata, so there is no second type system.

The glossary entry *Cell system, Scale versus the typical* carries the comparison with a conventional class-based system and the Alan Kay / Lisp framing; this note carries the design.

The surface is the Prose sentence grammar, not tensor shapes. The tensor entries in the image-encoding section are internal storage. A user never writes a tag or a tensor; they write a sentence. The exact grammar for class and slot declarations is a later decision, so the sentences below are the intended reading, not an admitted corpus yet:

1. "Every player is a character."
2. "Every character has health."
3. "Every player that is alive can jump."

The image encoding is how the reader stores the result. The surface is the contract; if the two ever diverge, the surface wins.

Design in one paragraph

A **class** is an image-resident metaobject that owns a name, a superclass list, a slot list, a metaclass, and a method-combination policy. A **slot** is an image-resident definition with a name, a value set, an optional default, and an initialization rule. An **instance** is a value with stable address identity and an appended state: the identity entry is written once, and every slot write appends a new state entry, exactly like channel state (D27). **Class precedence** is a deterministic linearization of the superclass graph; **dispatch** on a class is nominal (class identity plus precedence), not extensional, so it is stable as instances are added. **Methods** gain a qualifier, and a **method combination** orders them. **Closures** are image-resident code cells with an image-resident environment chain, so lexical scope does not become host state. **Metaclasses** make classes instances of a class, with a finite class-of closure (the cell query is `cell-class`) terminating at a root metaclass.

Compared with Smalltalk, CLOS, and Ruby/Python

The glossary carries the compact comparison. This section names the load-bearing differences. The comparison is against the design; stages A–G landed as D44–D50.

System	Shared	Different
Smalltalk	classes are cells; late-bound messages; a live image; blocks/ closures	append-only image; generic functions with multiple dispatch; no host cell graph; no `self`/`super` or `become:`; redefinition appends
CLOS	generic functions; multiple dispatch; method combination; the MOP	fixed meta-level, no reflective tower; append-only image, not mutable instances; no Lisp macros or condition system yet; a verification boundary

System	Shared	Different
Ruby	late binding; open definitions in spirit; blocks/procs; single inheritance plus modules	classes are image entries, so redefinition appends rather than reopens; dispatch is multiple and generic, not receiver method lookup; no `method_missing` silent fallback
Python	multiple inheritance; a deterministic precedence; metaclasses; closures	nominal classes and C3 class precedence; declared slots only, no `__dict__`; multiple nominal dispatch, not single-dispatch on the first argument; no attribute fallback

The four differences that matter:

1. **Image discipline.** Smalltalk, CLOS, Ruby, and Python put cells in a mutable host heap. Scale puts them in an append-only tensor image. A redefinition is a new entry; an instance's state is appended; history can roll back without rewriting the past.
2. **Dispatch.** Smalltalk, Ruby, and Python select a method from the receiver's class (or MRO). Scale uses image-resident generic functions over all arguments, with class identity and precedence for class specializers and value sets for structural specializers. CLOS is the closest here; Scale's difference is the image and the fixed meta-level.
3. **Meta-level.** CLOS has an open MOP; Smalltalk has a metaclass tower; Python has metaclasses and descriptors. Scale keeps a fixed meta-level and a finite `cell-class` closure. Classes, methods, combinations, and metaclasses are image data, but the evaluator is not a reflective tower.
4. **State and failure.** Their instances are mutable cells with attribute dictionaries, instance variables, or slots. Scale's instance is stable identity plus appended state; a missing slot is `MissingSlot`, an unbound slot is `UnboundSlot`, a wrong value is `SlotTypeMismatch`, and an undecidable dispatch is `DispatchUndecided`, never a silent `nil`/`method_missing`/`__getattr__` fallback.
5. **Verification.** The cell system is a runtime feature. It is not part of the SMT theory; the proof-or-witness boundary stays with the verification pipeline. That boundary is not a claim any of the four makes.
6. **Surface.** Their class declarations are code in a host syntax. Scale's declared surface is the Prose sentence grammar; the tensor entries in the design note are internal encodings, never syntax. Closures currently use the lab words `let`, `lambda`, and `apply`.

Image encoding (internal; never surface syntax)

The tags are proposals. The registry is append-only, and `ClassTag = -12` is already the classification verdict, so it is not reusable. A decision record must allocate the numbers when the layer lands.

Entry	Proposed tag	Internal entry
Class definition	`-15`	`[tag; name; metaclass; superclass…; slot…; combination?]`
Slot definition	`-16`	`[tag; name; value set; default?; initialization]`
Instance identity	`-17`	`[tag; class; instance number]`, written once
Instance state	`-18`	`[tag; instance; slot; value…]`, appended per write
Closure	`-19`	`[tag; quotation; environment]`
Environment frame	`-20`	`[tag; parent; name; value…]`

Entry	Proposed tag	Internal entry
Method combination	`-21`	`[tag; name; qualifier order; policy]`
Qualified method	`-22`	`[tag; qualifier; arity; body; specializer…]`
Metaclass	`-23`	`[tag; name; superclass…; slot…]`

The existing `GenericTag = -4` and `MethodTag = -5` stay readable. A qualified method is a new tag, so old images and old primary methods keep their shape. The generic function grows an optional combination reference; a missing field reads as `Standard`.

New roots are execution-state pointers, re-threaded on persistence like `SetsRoot`: a class root for name to class, and an instance-state root for instance to latest state. The instance state root is latest-write-wins; a write is an append, never an overwrite.

Semantics

Classes and slots (landed as D46)

`\"ClassName\" \"superclassNames\" \"slotNames\" class` declares a class; the lists are comma-separated and empty text means none. `valueSetName \"slotName\" slot` declares a slot. The declarations are the lab surface; the Prose sentence grammar is still deferred. Slot definitions are global by name for now, and the value set is resolved by name when the slot is used.

A class name is nominal. Two classes with equal slots are not the same class. A subclass's effective slots are its own slots followed by its superclasses' slots, deduplicated by name; a cycle is skipped. A deterministic precedence list is D47. A subclass may narrow a slot's value set, and an incompatible redefinition is a named `SlotConflict`. A slot's type is a runtime value set, not the verification `TypeLattice.FieldType`; the cell system and the verification lattice are separate coordinate systems (D32).

Reading an unbound slot is `UnboundSlot`; a missing slot is `MissingSlot`; a value outside the slot's set is `SlotTypeMismatch`; an undecidable predicate is `Undecided`, never a silent accept.

Instances and mutation (landed as D46)

`\"ClassName\" create-instance` creates a stable instance identity and an initial state. `slot-value` reads the latest state; `set-slot-value` validates the value against the slot's value set and appends a new state entry. The old state stays in the image. A missing slot is `MissingSlot`, an unbound slot is `UnboundSlot`, a value outside the set is `SlotTypeMismatch`, and an undecidable predicate is `ValueSetUndecided`.

The image is append-only, so a write cannot rewrite the instance. The design is identity plus appended state: the instance identity is stable, each write appends a state entry, and a root resolves the latest state. This is the channel-state pattern (D27) and preserves cell identity, at the cost of unbounded history. The append-only image has no garbage collection, so a heavily mutated long-lived cell grows without bound. If that becomes unacceptable, the fallback is immutable values, or an explicit compaction decision; neither is a silent rewrite.

Class precedence (landed as D47)

Class precedence is a C3 linearization of the superclass graph, most-specific first and including the class itself. A cycle is `ClassCycle`; a conflicting local order is `InconsistentPrecedence`. `class` validates the new class before it enters the class root, so a bad graph never becomes the latest

definition. Precedence is computed from image data; a cache is derived execution state, never serialized. `class-precedence` exposes the list.

Dispatch (landed as D47)

An instance's class is its class pointer. A class specializer applies when the instance's class is that class or a subclass; specificity is precedence inclusion. Value-set specializers keep their extension-inclusion rule. A class and a value set are incomparable, so a tie is `AmbiguousDispatch`, never a silent pick; no applicable method is `NoApplicableMethod`. `subclass-of` and `instance-of` answer the class queries.

Method combination

Landed as D48. The qualifiers, effective order, `call-next-method`, `next-method-p`, and `NoNextMethod` are applied; the chain and its continuations are execution state. A method qualifier is Primary, Before, After, or Around. The effective order is Around most-specific-first, Before most-specific-first, Primary most-specific-first, and After least-specific-first. `call-next-method` runs the next method in the effective chain; no next is `NoNextMethod`; `next-method-p` answers a verdict. The chain is control state in a call frame, not serialized. The combination policy is per generic function, with `Standard` as the default; a new combination is data, not an F# case.

Closures and lexical scope (landed as D45)

`let` creates a child frame and makes it current; `lambda` captures the current frame in an image-resident closure; `apply` on a closure runs its quotation in the captured frame and restores the caller's frame. `lookup` walks the frames, then the global binding root; no host `Map` environment enters `EvaluationState`. `define` and `bind` stay global; a bare quotation applied with `apply` keeps dynamic scope, exactly as before. The current frame and the saved caller frames are execution state, never serialized; an internal return token restores the caller, and a return without a call is `UnmatchedReturn`. A `QuotationRun` probe resets frames, a spawned actor starts at the global frame, and history rollback returns to the global frame (call-frame capture in the event stream is a later change).

Metaclasses

Landed as D49. `metaclass`, `use-metaclass`, and `cell-class` are applied; a user metaclass needs no F# case, and the closure terminates at `StandardClass`. A class refers to a metaclass, with `StandardClass` as the default. The cell query (not the old `class-of` classification word) answers: instance to class, class to metaclass, metaclass to the root metaclass, root to itself. The closure is finite and terminates at the root, as D16 requires; it is not a reflective tower. A minimal metaclass supports class-level slots and allocation; the full CLOS metaclass protocol is a later refinement.

Invariants and named refusals

The layer must preserve the house rules: image-only data; roots are execution state; no host `Map` environments; failures are values with named reasons; an undecidable slot type or dispatch is `Undecided`, never false; no `private` outside a smart constructor; the append-only image is never rewritten; a protocol decision returns a verdict, never a silent pick.

Refusals: `UnknownClass`, `UnknownSlot`, `MissingSlot`, `UnboundSlot`, `SlotTypeMismatch`, `SlotConflict`, `DuplicateSlot`, `ClassCycle`, `InconsistentPrecedence`, `NotAnInstance`, `NotAClass`, `NoNextMethod`, plus the existing `AmbiguousDispatch` and `NoApplicableMethod`.

Hard tensions

1. **Append-only versus mutation.** State-versioned instances preserve identity but grow without bound; immutable instances fit the image but change the cell model. D46 chose identity plus appended state; the growth policy for long-lived mutation remains open.
2. **Nominal versus structural.** Classes are nominal; value sets are structural. Dispatch must not conflate them.
3. **The level model.** An instance is a base value that carries a class reference. D16′s "levels at or below" rule governs metaobject-to-metaobject dependencies; ordinary values may carry metaobject references as data. State that clarification in the encoding decision.
4. **Naming.** `class` is the cell-system term; the old `ClassTag` and `class-of` are now `ValueTag` and `value-of`. The cell query is `cell-class` to avoid reviving the old meaning.
5. **Closures before combination.** Closures landed as D45; method combination landed as D48, so Around methods can capture lexical state.
6. **Conditions are values.** D50 makes a protocol failure a condition value; `with-handler`, `signal`, and `invoke-restart` are the protocol, and handler frames are execution state, never serialized.
7. **Performance.** Precedence, method selection, and slot lookup need derived caches in execution state, not host data absent from the image.
8. **NativeAOT.** Everything stays data-driven and reflection-free.

Stages

Stages A–G landed as D44, D45, D46, D47, D48, D49, and D50. The numbers are indicative.

Stage	Work	Decision	Acceptance
A	Encoding, tags, roots, persistence (landed)	D44	Classes, slots, instances, state, and closures round-trip; no host metaobject store.
B	Closures and lexical scope (landed)	D45	A closure captures a binding; `apply` sees it; no host map.
C	Slots and instances (landed)	D46	Class, slot, create-instance, slot-value, set-slot-value; identity stable; named refusals; persistence.
D	Class precedence and nominal dispatch (landed)	D47	Deterministic precedence; cycles refused; instance-of and subclass-of verdicts; most-specific class method; ambiguity refused.
E	Method qualifiers and combination (landed)	D48	Before, after, and around order pinned; call-next-method; NoNextMethod; a new combination needs no F#.
F	Metaclasses and the cell-class closure (landed)	D49	Finite closure; root metaclass self-describing; a user metaclass needs no F#.
G	Conditions and restarts (landed)	D50	Protocol failures are values or witnesses; Repair can consume them.

All stages are landed. The condition layer is data plus dynamic handler frames; it does not add an exception path to the evaluator.

Testing

Examples for every operation and refusal. Properties for precedence determinism over random acyclic class graphs, cell-class closure termination, slot write and read round-trip, method-

combination order, closure capture, persistence with roots re-threaded, and the absence of host state outside the image. A small reference cell model supplies differential checks for dispatch and slot lookup. granular wrappers and the count update follow the existing suite; the gate is `just check`.

Recommendation

Do it as a separate prepared vertical slice, in the same spirit as the decision tables and the DSPy track: no kernel change until the slice lands, no new dependency, and no change to a domain that does not use cells. The two decisions that determine everything else are instances as identity plus appended state (with an explicit growth policy), and closures before method combination. Closures landed first as D45.